

C Programming

Environment Setup & Ch.1 Programming Concept

What You Will Learn in This Chapter

Ch.1 Programming Concept

- Programming Concepts
- Programming language
- **Algorithm**
- Before writing a program, let's review some important concepts.

Getting your computer ready to write C code

Environment Setup

What Do You Need?

Tools

- A text editor or IDE to write code, and a C **compiler** to translate it into a program
- We will use GCC (GNU **Compiler** Collection)
- Recommended: Visual Studio Code + MinGW (Windows) / Xcode Command Line Tools (macOS)
- No install needed: an online **compiler** such as OnlineGDB (<https://www.onlinegdb.com/>)

Description of the First Program

```
#include <stdio.h>

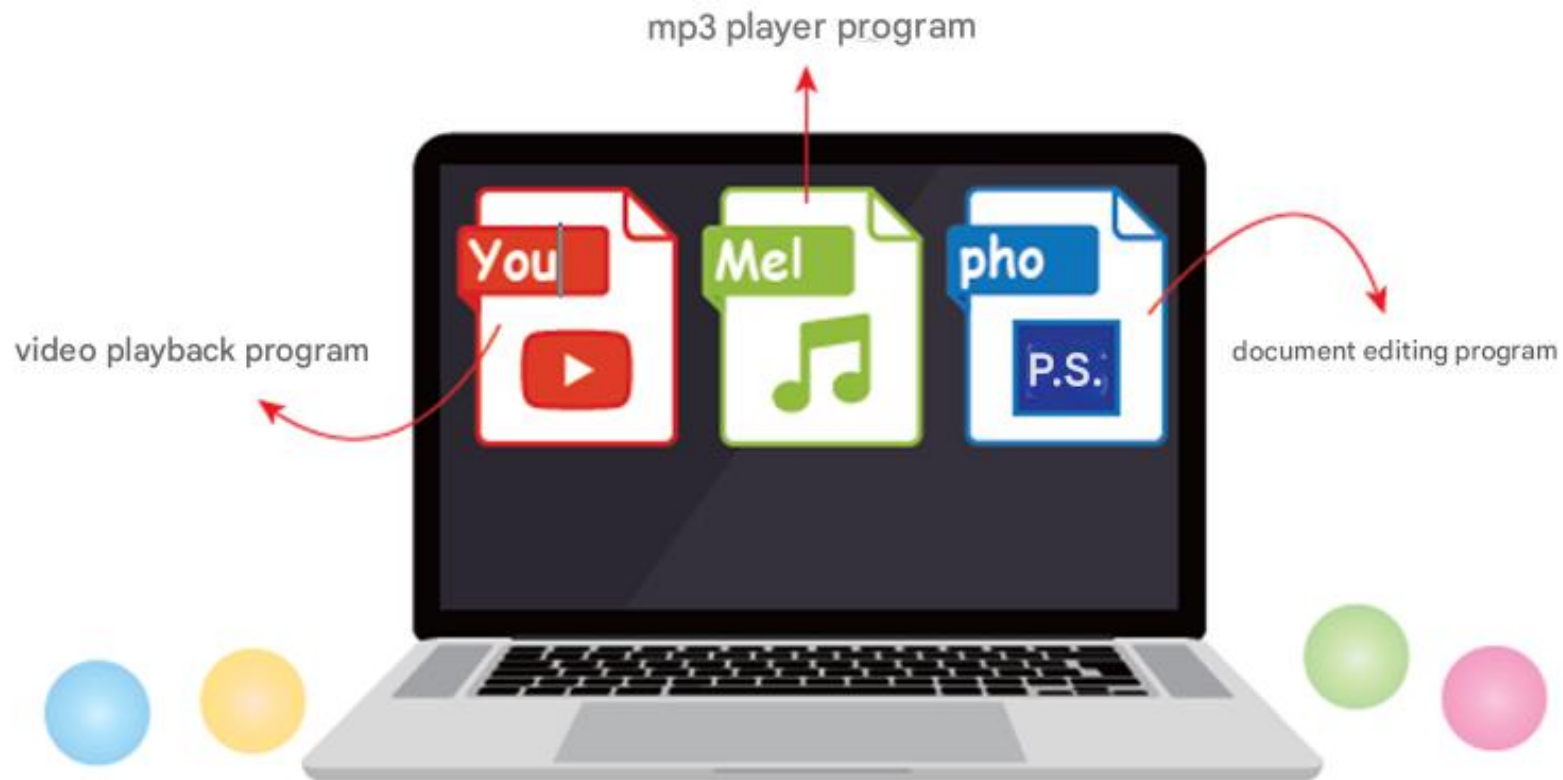
int main (void)
{
    printf( "Hello World!" ) ;
    return 0;
}
```

hello.c — compile, then run it to see the output.

What Is a Program?

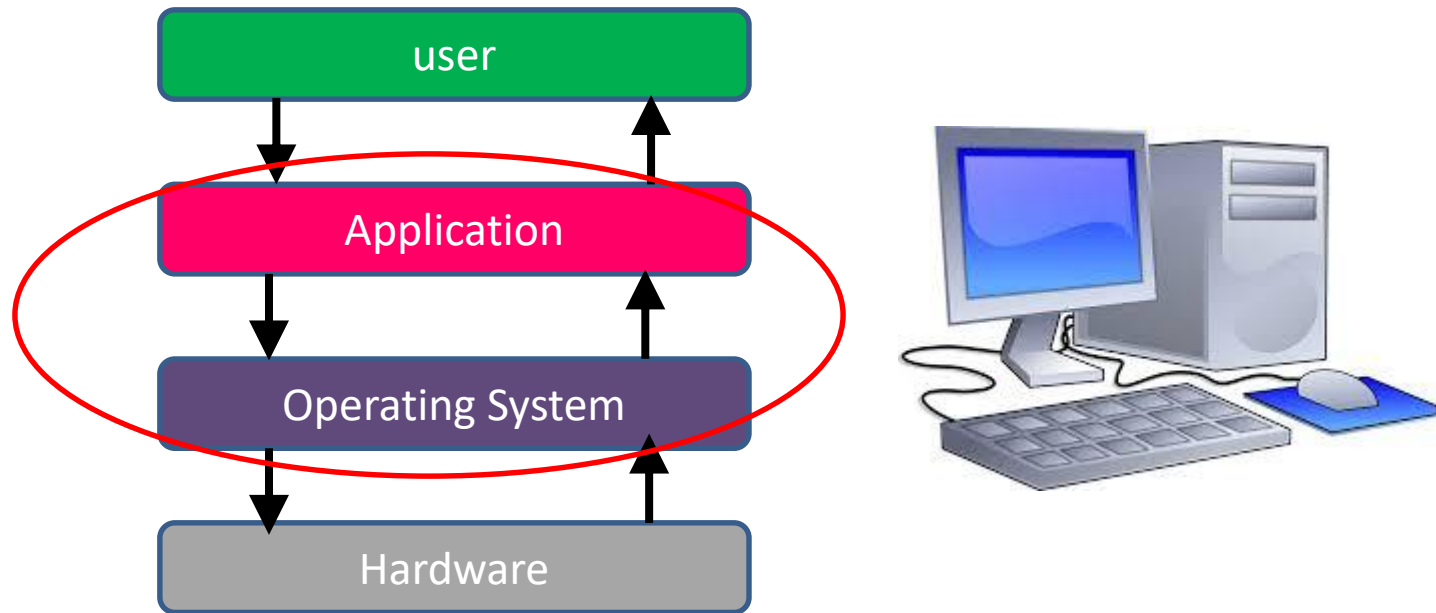
What Is a Program?

- A computer is a general-purpose machine -> you can do many things using a computer.
- What makes computers general-purpose is the concept of programs.



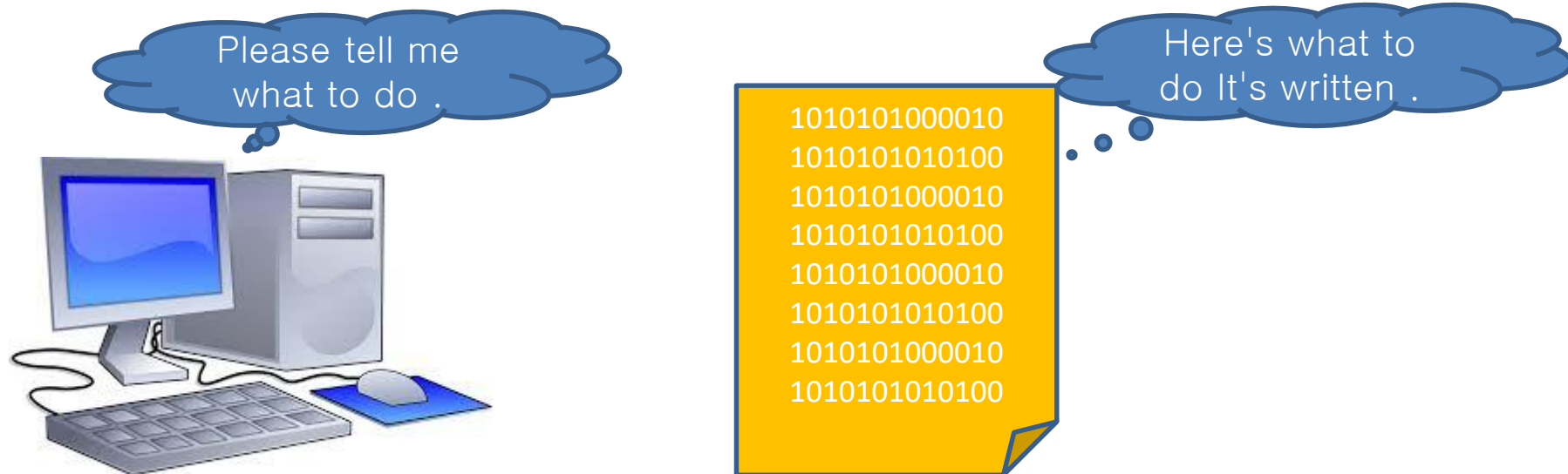
What If There Is No Program on the Computer?

- Without computer hardware and no programs, a computer is just a useless machine that generates some heat and noise.
- Layers: user -> Application -> Operating System -> hardware.



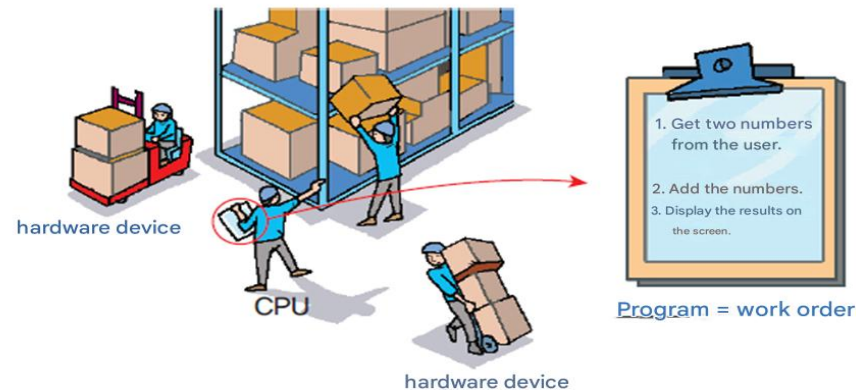
Q) Why Don't Computers Just Work Like Home Appliances?

- Q) Why don't computers operate right away like home appliances without installing programs, but instead require users to install programs, which is inconvenient?
- A) Computers need programs because they are made to do **many different jobs**. Home appliances, like washing machines, have only a few jobs. Their programs are already installed in the factory, so we can use them right away. But computers can be used for games, work, studying, editing videos, and much more. So users install the programs they need.



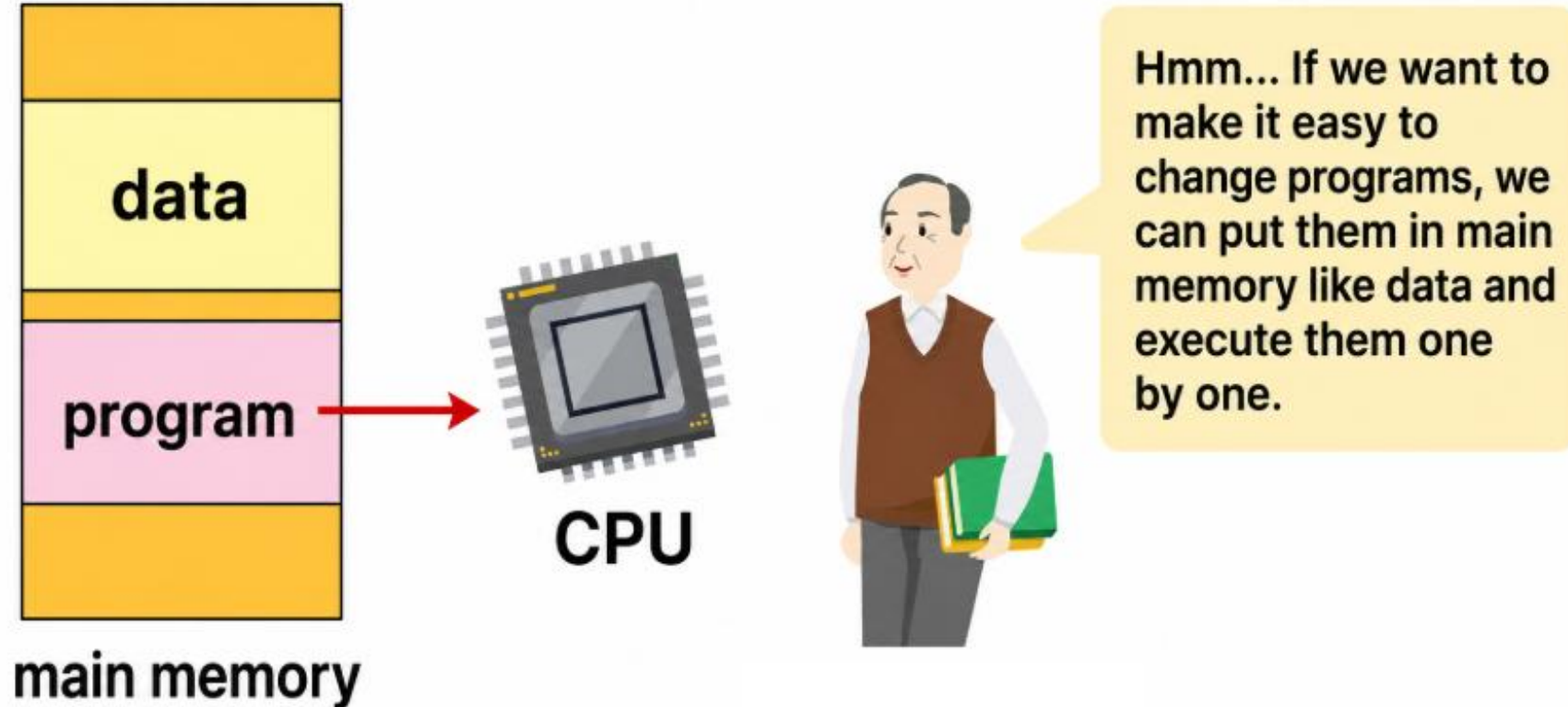
What's Inside the Program?

- A program can be thought of as a **work instruction for a specific task**.
- To perform a task, you need to list **instructions**. A program contains instructions. These instructions are called commands.
- Example: a program that reads three numbers and calculates their average is made up of a list of such instructions.
- Program == Work Instructions: a program is a document that tells the computer what to do.
- Because they lack common sense or intelligence, they must be given very detailed and specific instructions on what to do .



Stored-Program Concept

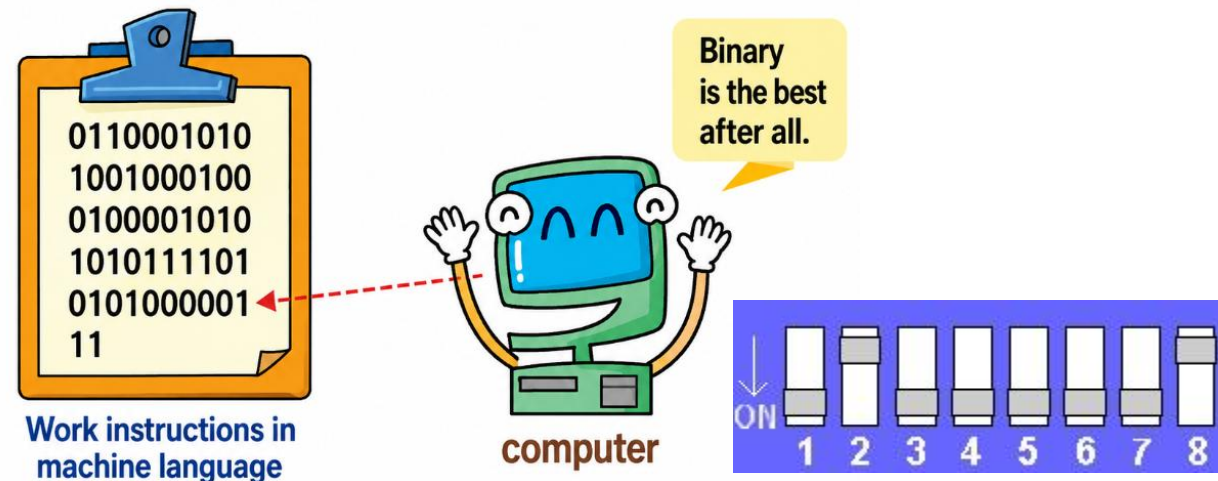
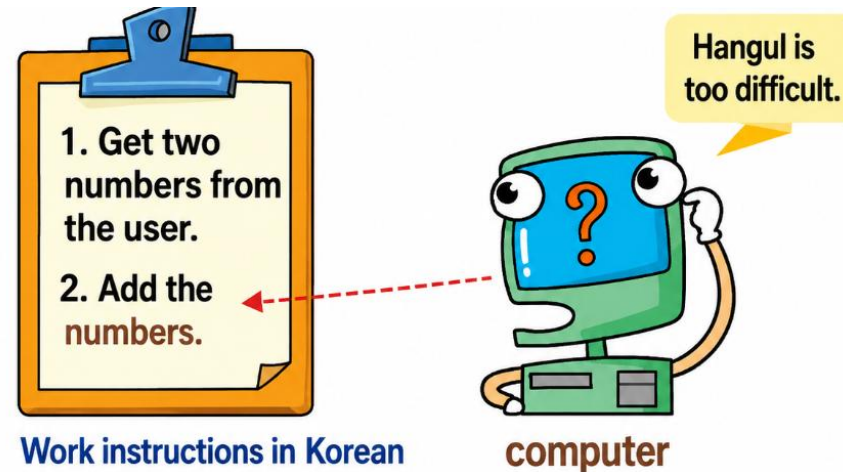
- Programs are stored in main **memory** -> can be easily changed
- Sequentially retrieves and executes commands from a program stored in main memory .



Programming Language

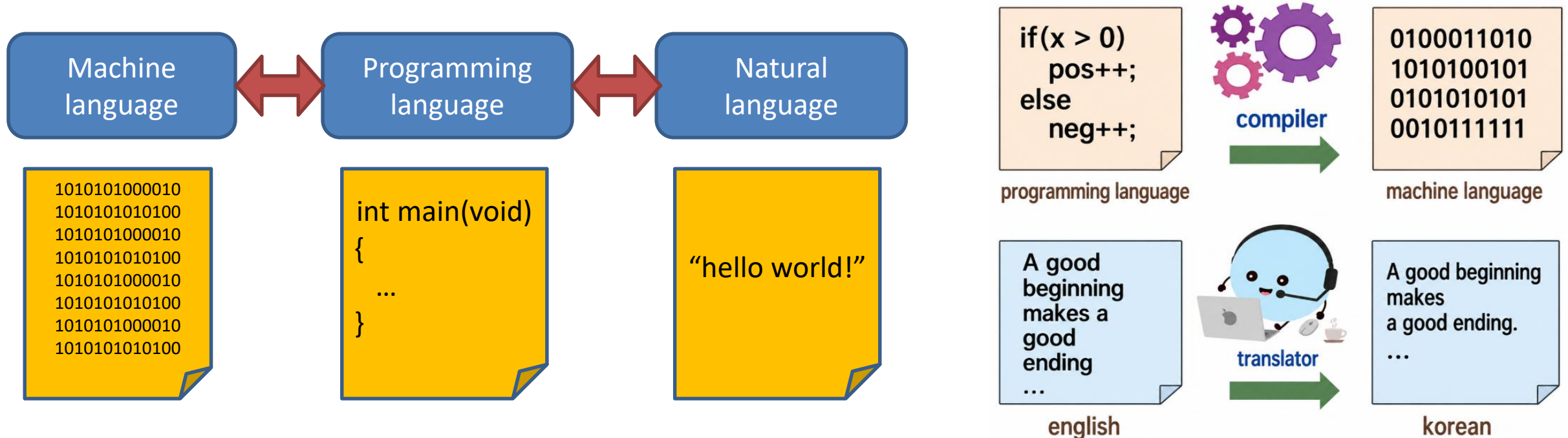
Language That Computers Understand

- Computers understand only one language: **machine language**, which consists of 0s and 1s, such as "001101110001010..."
- Computers express everything as 0 and 1, and work by turning internal switch circuits ON/OFF based on 0 and 1.



Programming Language

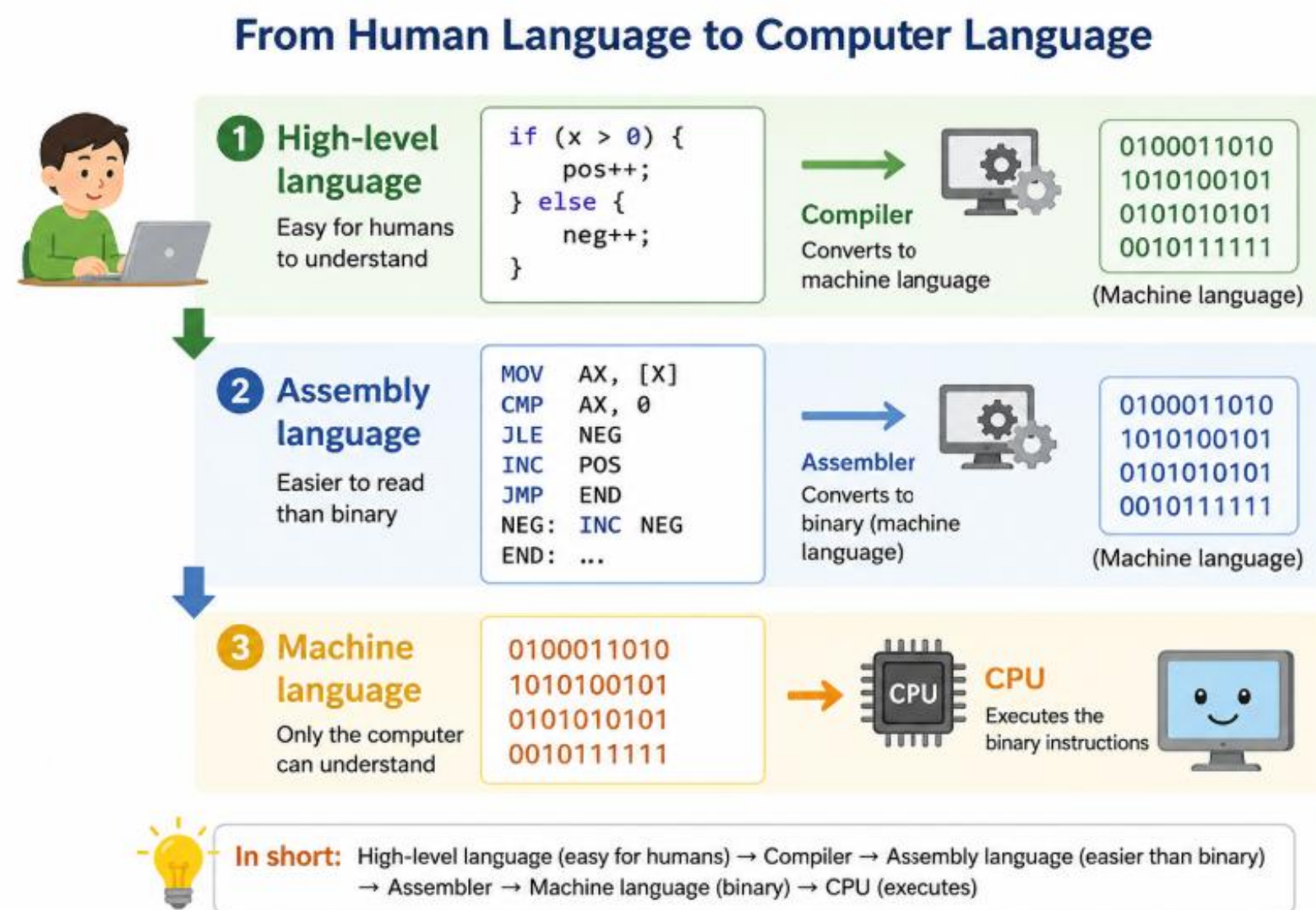
- Although **machine language** can be used, it is very inconvenient because programs must be written in binary.
- Programming languages are somewhere between natural language and **machine language**.
- A **compiler** translates a programming language into **machine language**.
- A **compiler** can be considered an interpreter between humans and computers.



Classification of Programming Languages

Level	Description
Machine language	A binary representation of the instructions of a specific computer. Consists of 0 and 1. Hardware dependent.
Assembly language	CPU commands expressed in symbols that are abbreviations of English letters rather than binary numbers. An assembler converts symbols into binary.
High-level language	Allows you to write programs independently of the architecture or processor of a specific computer (C, C++, JAVA, FORTRAN, PASCAL). A compiler converts high-level language statements into machine language.

From Human Language to Computer Language



Type of high-level languages

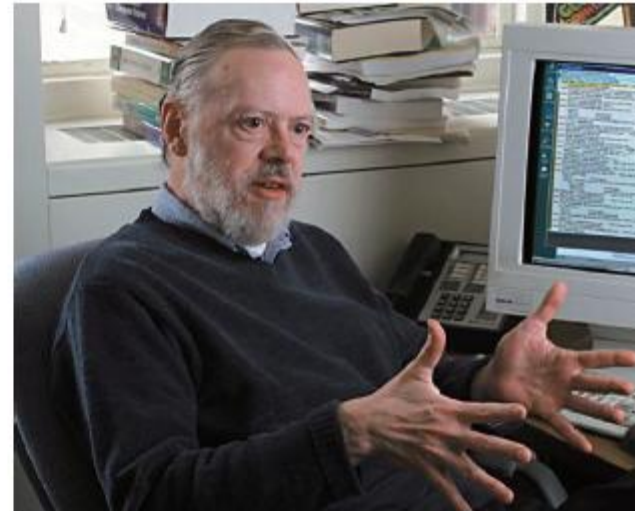


Language	Characteristics	“Hello World” Example
FORTRAN	Created in the 1950s . Mainly used for numerical and scientific computing .	<pre>PROGRAM HELLOPRINT '(A)', 'Hello World'STOPEND</pre>
COBOL	A business processing language created in 1959 . Designed to use a sentence-like format.	<pre>IDENTIFICATION DIVISION.PROGRAM-ID. HELLO- WORLD.PROCEDURE DIVISION.DISPLAY 'Hello World'.STOP RUN.</pre>
Python	Developed by Guido van Rossum in 1991 . Easy to learn and widely used in AI and data science .	<pre>print("Hello World")</pre>
C	Created by Dennis Ritchie in the early 1970s at Bell Labs. Widely used for systems programming .	<pre>int main(void){printf("Hello World\n");return 0;}</pre>
C++	Developed by Bjarne Stroustrup at Bell Labs in 1983 . Adds object-oriented programming features to C.	<pre>int main(){cout << "Hello World" << endl;return 0;}</pre>
Java	An object-oriented language developed by James Gosling at Sun Microsystems in 1995 .	<pre>public class Hello {public static void main(String[] args) {System.out.println("Hello World");}}</pre>

Introduction of C Language

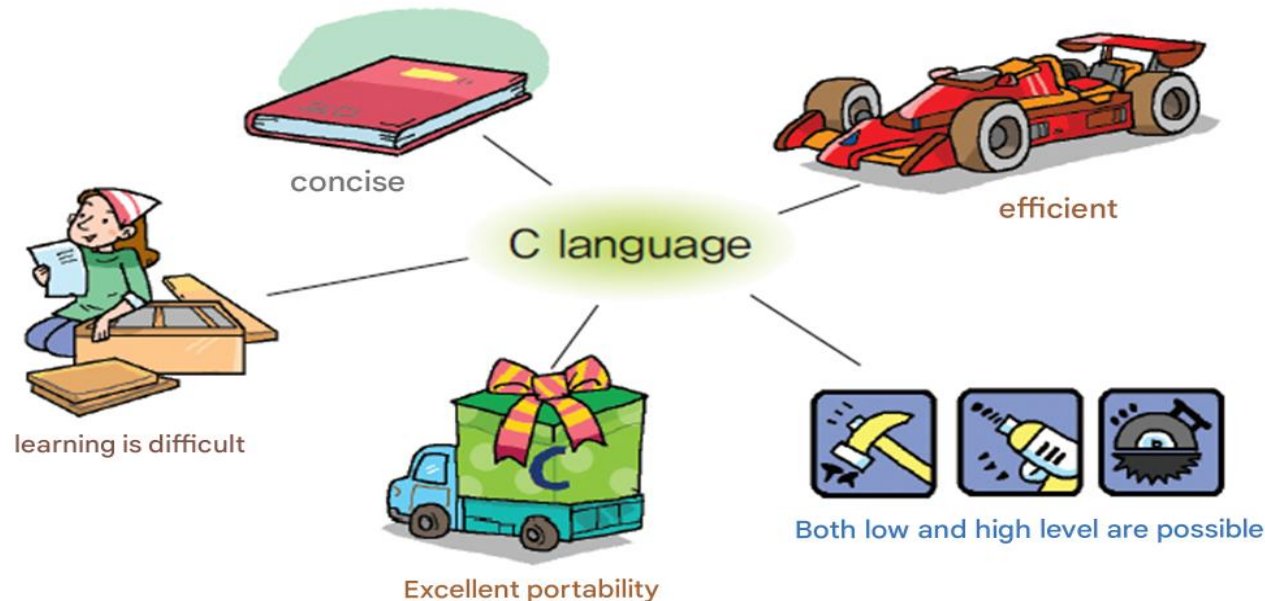
Introduction of C Language

- Developed by Dennis Ritchie of AT&T in the early 1970s.
- Evolved from an earlier language called B, into C.
- Created for the development of the UNIX operating system.



Features of the C Language

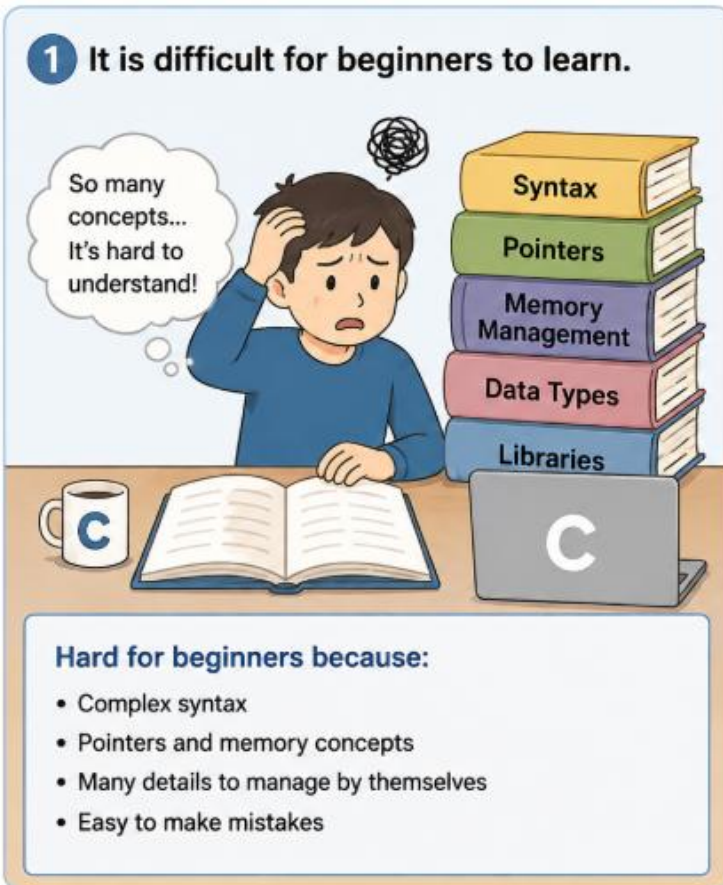
- It's concise, and it is efficient.
- The C language allows for both low-level programming that directly controls hardware, as well as high-level programming.
- The C language is highly portable.
- The C language will help you understand how computer hardware works.



Disadvantages of the C Language

- It is difficult for beginners to learn.
- There are many cases where pointers, which are essential elements for controlling hardware, are misused.

1 It is difficult for beginners to learn.



So many concepts... It's hard to understand!

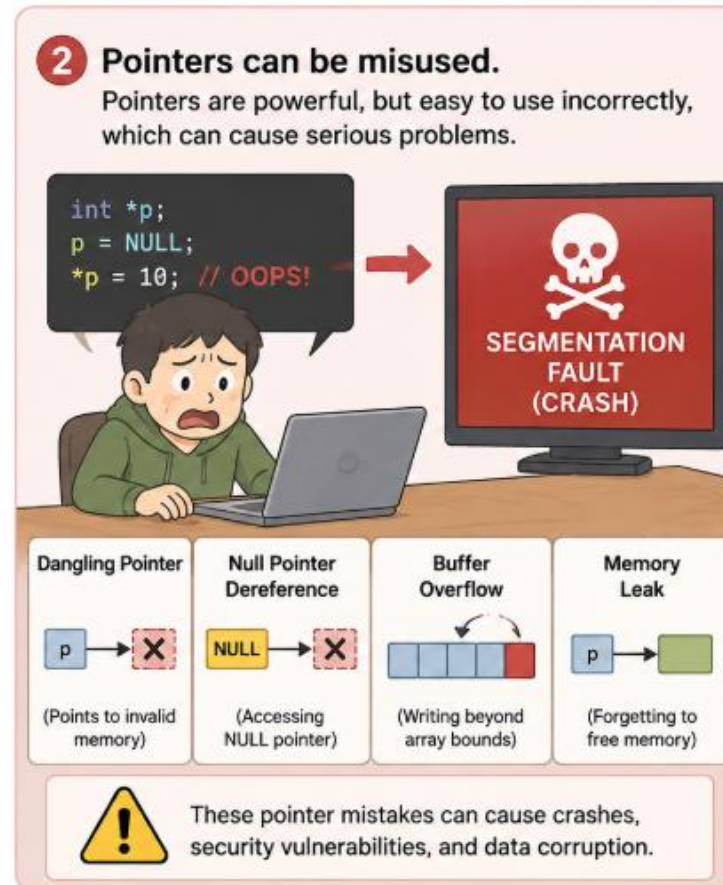
Syntax
Pointers
Memory Management
Data Types
Libraries

Hard for beginners because:

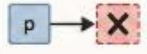
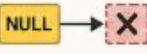
- Complex syntax
- Pointers and memory concepts
- Many details to manage by themselves
- Easy to make mistakes

2 Pointers can be misused.
Pointers are powerful, but easy to use incorrectly, which can cause serious problems.

```
int *p;  
p = NULL;  
*p = 10; // OOPS!
```



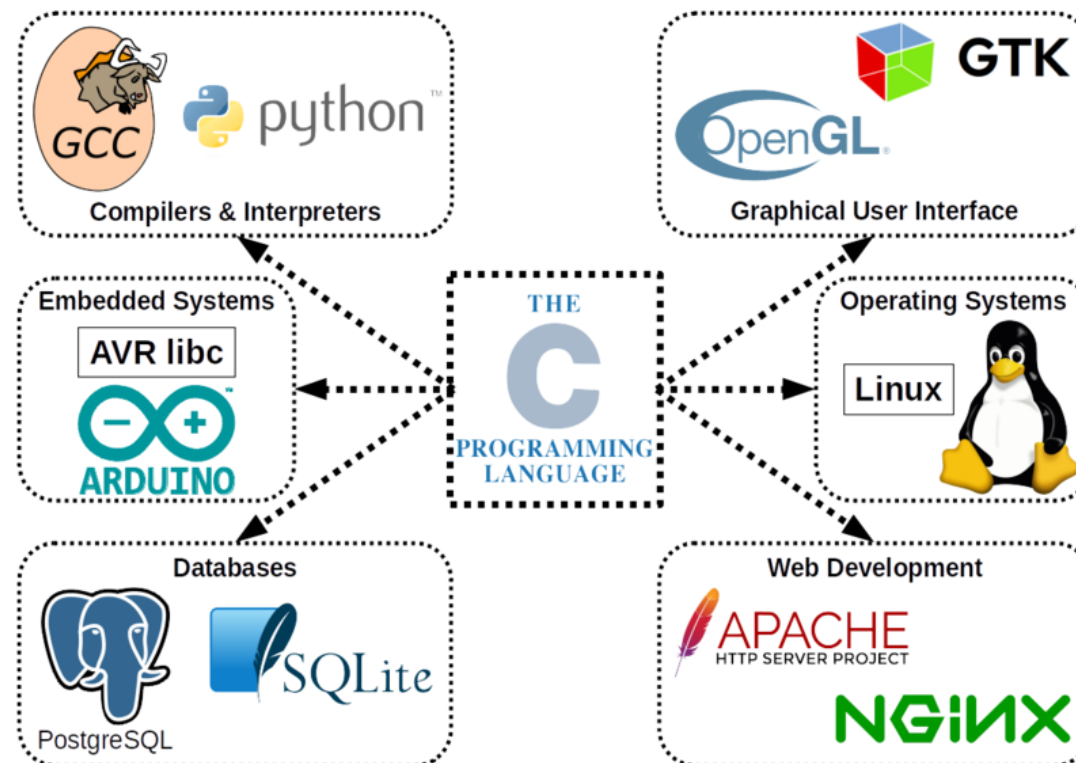
SEGMENTATION FAULT (CRASH)

Dangling Pointer	Null Pointer Dereference	Buffer Overflow	Memory Leak
			
(Points to invalid memory)	(Accessing NULL pointer)	(Writing beyond array bounds)	(Forgetting to free memory)

! These pointer mistakes can cause crashes, security vulnerabilities, and data corruption.

Will the C Language Still Be Used in the Future?

- The C language is a common part of C++ and JAVA.
- Programs where execution speed is important are implemented in the C language.
- The C language is widely used in **embedded systems**: a special-purpose system in which a computer is embedded in a device, such as an MP3 player or a cell phone.
- Uses of C Language



Algorithm

Algorithm

Q) Can anyone cook if they just learn how to use an oven and have the ingredients?

A) You need to know HOW to cook. In the same way, a computer needs an **algorithm**.

- What is an **algorithm**? An **algorithm** is a description of the step-by-step procedure that a computer must perform to solve a problem.
- Example: consider the problem of finding the phone number of a specific person (say, Park Cheol-su) in a phone book.



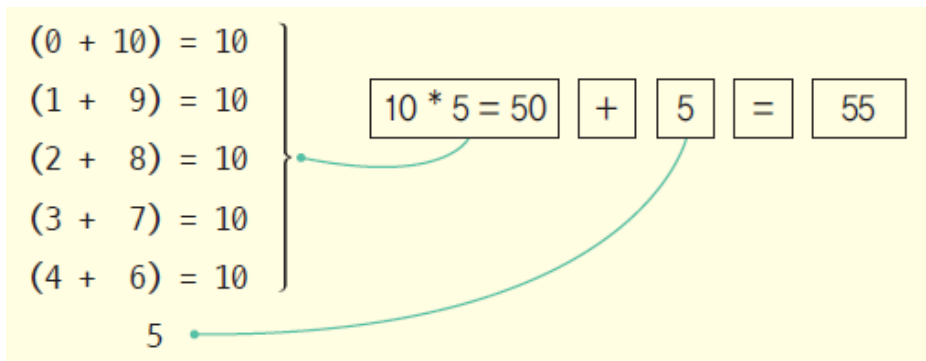
Example: An Algorithm for Making Bread

- 1. Prepare an empty bowl.
- 2. Add yeast to flour and milk, and stir.
- 3. Add butter, sugar, and eggs, and mix.
- 4. Leave in a warm place to ferment.
- 5. Bake in an oven.



Example: An Algorithm to Find the Sum of Numbers from 1 to 10

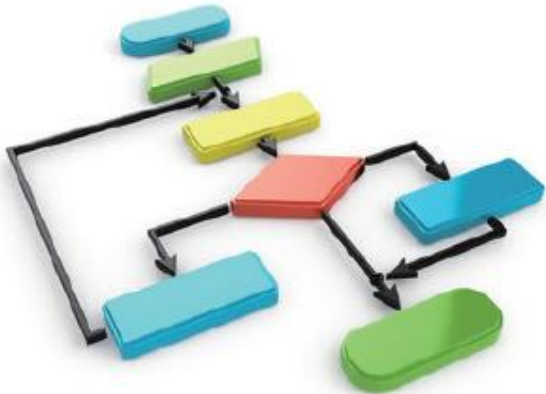
- Way 1: add the numbers from 1 to 10 one by one: $1 + 2 + 3 + \dots + 10 = 55$
- Way 2: group the numbers so that the sum of each pair is 10, multiply the number of groups by 10, and add the remaining number 5.



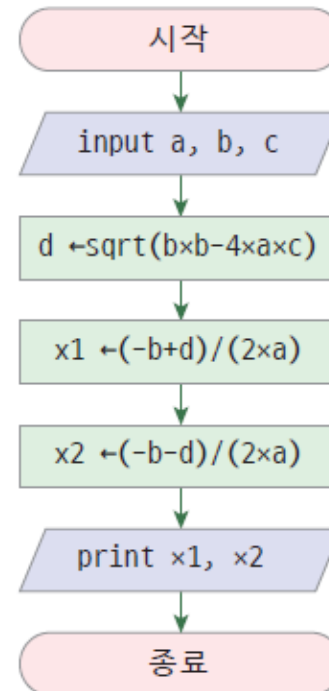
- Way 3: calculate it using the formula: $10(1+10)/2 = 55$

The Art of Describing Algorithms

- **Natural language** - plain sentences describing the steps.
- **Pseudocode** - code that is more systematic than natural language and less rigorous than a programming language, mainly used to express algorithms.
- **Flowchart** - a method of graphically representing the logical sequence of operations in a program, using boxes and arrows.



```
total ← 0
counter ← 1
while counter ≤ 10
    input grade
    grade ← grade + total
    counter ← counter + 1
average ← total / 10
print average
```



- Step 1: input a, b, c
- Step 2: $d \leftarrow \sqrt{b \times b - 4 \times a \times c}$
- Step 3: $x1 \leftarrow (-b + d) / (2 \times a)$
- Step 4: $x2 \leftarrow (-b - d) / (2 \times a)$
- Step 5: print x1, x2

Pseudocode Example

```
BEGIN  
    Take the grades of 10 students  
    Calculate the average  
    PRINT the average  
END
```

The algorithm that takes the grades of 10 students and calculates the average, expressed in pseudocode.

The Importance of Algorithms

- If we give computers bad algorithms, we are bound to get bad results.
- Many errors in our smartphones, home appliances, and cars are caused by bad algorithms.
- If there are no errors in the **algorithm**, the computer program will also operate without logical errors.

How to Create an Algorithm

- Rather than solving a problem all at once, break it down into smaller problems.
- Keep breaking the problem down until it is small enough.

- ① Clean the room.
- ② Clean the living room.
- ③ Clean the kitchen.

- ① Ventilate.
- ② Organize your things
- ③ Turn on the vacuum cleaner
- ④ Mopping .



1. Dusting

Remove dust and dirt.



2. Cleaning

Wipe surfaces and remove stains.



3. Vacuuming

Clean the floor and remove dust.



4. Mopping

Make the floor clean and shiny.

Summary

- **A program** is a set of work instructions that a computer executes.
- Programming languages sit between natural language and **machine language**; a **compiler** translates them into **machine language**.
- C is a high-level, efficient, portable language - powerful, but with a learning curve (especially pointers).
- Every good program starts from a clear **algorithm**, expressed as natural language, **pseudocode**, or a **flowchart**.

Q & A

Lab & Quiz: right after this - see you in the practice session!